

Programación I - Primer Semestre 2026
Trabajo Práctico: Super Elizabeth Sis contra el ataque de los peces del infierno

Las Autoridades de la UNGS nos encargaron desarrollar un video juego similar al Super Mario Bros pero con algunas importantes diferencias que mencionamos a continuación.

El malvado Rey Camir, al frente de la patrulla del maltrato animal, secuestró al pez Remo, la mascota favorita de la princesa Elizabeth. La princesa Elizabeth transformada ahora en nuestra heroína se lanza a combatir a los peces del infierno, que son copias malvadas de su pez favorito, el cual deberá rescatar (ver Figura 1).

El objetivo de este trabajo práctico es crear un simulador en el cual la princesa Elizabeth vaya avanzando en un escenario, en el que aparecen distintos enemigos que tiene que eliminar y además evitar caer al precipicio.



Figura 1: Ejemplo del juego.

El Juego: Super Elizabeth Sis contra el ataque de los peces del infierno

Requerimientos obligatorios

1. Al iniciar el juego, la princesa Elizabeth debe aparecer en el sector izquierdo de la pantalla (ver Figura 1). El escenario del juego consiste en el piso que tiene algunos huecos (al menos 2) y también islas flotantes (al menos 3).
2. Mientras la princesa esté en sobre el piso o sobre una isla flotante: Si se presiona la

fecha hacia atrás, la princesa debe retroceder. Si se presiona la fecha hacia adelante, la princesa debe avanzar. Si no se presiona ninguna tecla, debe «permanecer quieta». Si se presiona la tecla hacia arriba debe saltar, así puede esquivar enemigos o subirse a una isla flotante. Cuando la princesa no esté sobre una isla o el piso, deberá estar en caída libre.

3. Movimiento del piso y las islas flotantes: a medida que pasa el tiempo tanto el piso, como las islas flotantes deberán moverse de derecha a izquierda con la misma velocidad, creando la ilusión de que el escenario esta avanzando. Si no se presiona ninguna tecla, la princesa debe moverse con el escenario. Pero si llega al borde izquierdo de la pantalla, debe permanecer allí y no salirse de la pantalla.
4. Peces del infierno: los peces deben aparecer aleatoriamente desde el extremo derecho de la pantalla, a distintas alturas por encima del suelo. Deben moverse hacia la izquierda con una velocidad diferente, tienen la capacidad de volar y atravesar objetos. Los peces no se pueden superponer entre ellos.
5. El fuego de Elizabeth: Cuando se presione el botón izquierdo del mouse, la princesa podrá lanzar una pequeña bola de fuego que se moverá en la dirección del mouse al momento del disparo. Solo se puede lanzar una bola de fuego a la vez. Es decir que si ya se lanzó una bola de fuego, no se puede crear otra hasta que la primera desaparezca o salga de la pantalla.
6. Cuando la bola de fuego haga colisión con un pez, lo elimina y ambos objetos deberán desaparecer. Si la bola de fuego hace colisión con el piso, o una isla flotante, solo la bola de fuego desaparece.
7. Ganar el juego: El juego se da por ganado cuando se elimina una cierta cantidad de peces usando las bolas de fuego. La cantidad queda a elección del grupo (se recomienda que sea entre 20 y 30).
8. Perder el juego: El juego se da por perdido cuando la princesa haga colisión con un pez o cuando cae al precipicio, es decir, cuando cae en los huecos del piso.
9. Cuando el piso, las islas flotantes y los peces que no son eliminados pasen el borde izquierdo de la pantalla, deberán hacerse nulos. No vale contar los peces que son eliminados de esta forma para ganar el juego.
10. Durante todo el juego deberá mostrarse en algún lugar de la pantalla la cantidad de peces eliminados, la cantidad restante para ganar el juego y alguna otra información que el grupo crea relevante.
11. El código del proyecto **deberá tener un buen diseño** de modo que cada objeto tenga **bien delimitadas sus responsabilidades**.

Aclaraciones: cuando un objeto ya no esta más debe ser eliminado (null), no vale únicamente ocultar las imágenes del juego.

Para la implementación de los peces se debe usar arreglos de objetos

Requerimientos opcionales

La implementación a entregar debe cumplir como mínimo con todos los requerimientos obligatorios planteados arriba, pero si el grupo lo desea, puede implementar nuevos elementos para enriquecer la aplicación. Sugerimos a continuación algunas ideas:

- Distintos tipos de peces: peces con la habilidad de esquivar disparos o de ir en dirección hacia donde esta la princesa o de tener un movimiento diferente.
- Disparos de los peces: Los peces arrojan bombas que se mueven hacia la derecha hasta que hagan colisión con la princesa. Esas bombas solo pueden ser destruidas por los disparos de fuego.
- Otra habilidad para la princesa: podrá lanzar un hechizo mágico que afecte a peces que esten en un radio cercano. El hechizo se activa cuando se presiona el botón derecho del mouse sobre un pez. Se deberá esperar un cierto tiempo para volver a activarlo.
- Items especiales: Regalos (o trampas) que aparecen aleatoriamente, al hacer colisión con un item, puede dar (o quitar) puntos o un disparo especial o alguna habilidad.
- Muchos niveles: cuando se elimina una cierta cantidad de peces, pueden aparecer peces mas resistentes, es decir que se necesite mas de un disparo para eliminarlo o mas rápidos.
- Pez colosal: un pez gigante como enemigo final mucho mas resistente, este pez deberá quedarse del lado derecho de la pantalla.

Informe solicitado

Además del código, la entrega debe incluir un documento en el que se describa el trabajo realizado, que debe incluir, **obligatoriamente y como mínimo**, las siguientes secciones:

Carátula: Una carátula del documento con nombres, apellidos, legajos, y direcciones de email de cada integrante del equipo.

Introducción: Una breve introducción describiendo el trabajo práctico.

Descripción: Una explicación general de cada clase implementada, describiendo las variables de instancia y dando una breve descripción de cada método.

También deben incluirse los problemas encontrados, las decisiones que tomaron para poder resolverlos y las soluciones encontradas.

Implementación: Una sección de implementación donde se incluya el código fuente correctamente formateado y comentado, si corresponde.

Conclusiones: Algunas reflexiones acerca del desarrollo del trabajo realizado y de los resultados obtenidos. Pueden incluirse lecciones aprendidas durante el desarrollo del trabajo.

Condiciones de entrega

El trabajo práctico se debe hacer en grupos de **exactamente tres** personas.

El nombre del proyecto de Eclipse debe tener el siguiente formato: los apellidos en minúsculas de los tres integrantes, separados con guiones, seguidos de **-tp-p1**.

Por ejemplo, **amaya-benelli-castro-tp-p1**.

Cada grupo deberá entregar tanto el informe como el código fuente del trabajo práctico.

La modalidad de entrega es mediante repositorio en Git: asegurarse de que el repositorio sea privado y que el nombre del proyecto de Git tenga los apellidos en minúsculas de los tres integrantes, separados con guiones, seguidos del string **-tp-p1**.

Por ejemplo, **amaya-benelli-castro-tp-p1**. Consultar el username de cada docente y agregar a los docentes como **maintainer's**. El informe del trabajo práctico en formato pdf puede incluirse directamente en el repositorio.

Fecha de entrega: Hasta el martes 26/5 a las 23:59 hs.

Apéndice: Implementación base

Junto con este enunciado, se les entrega el paquete `entorno.jar` que contiene la clase `Entorno`. Esta clase permite crear un objeto capaz de encargarse de la interfaz gráfica y de la interacción con el usuario. Así, el grupo sólo tendrá que encargarse de la implementación de la inteligencia del juego. Para ello, se deberá crear una clase llamada `Juego` que respete la siguiente ¹signatura:

```
public class Juego extends InterfaceJuego {
    private Entorno entorno;

    // Variables y metodos propios de cada grupo
    // ...

    public Juego() {
        // Inicializa el objeto entorno
        entorno = new Entorno(this, "Proyecto para TP", 800, 600);

        // Inicializar lo que haga falta para el juego
        // ...

        // Inicia el juego
        entorno.iniciar();
    }

    public void tick() {
        // Procesamiento de un instante de tiempo
        // ...
    }

    public static void main(String[] args) {
        Juego juego = new Juego();
    }
}
```

El objeto `entorno` creado en el constructor del `Juego` recibe el juego en cuestión y mediante el método `entorno.iniciar()` se inicia el simulador. A partir de ahí, en cada instante de tiempo que pasa, el entorno ejecuta el método `tick()` del juego. Éste es el método más importante de la clase `Juego` y aquí el juego debe actualizar su estado interno para simular el paso del tiempo. Como mínimo se deben realizar las siguientes tareas:

- Actualizar el estado interno de todos los objetos involucrados en la simulación.
- Dibujar los mismos en la pantalla (ver más abajo cómo hacer esto).
- Verificar si algún objeto aparece o desaparece del juego.

¹**Importante:** las palabras clave “`extends InterfaceJuego`” en la definición de la clase son fundamentales para el buen funcionamiento del juego.

- Verificar si hay objetos interactuando entre sí (colisiones por ejemplo).
- Verificar si los usuarios están presionando alguna tecla y actuar en consecuencia (ver más abajo cómo hacer esto).

Para dibujar en pantalla y capturar las teclas presionadas, el objeto **entorno** dispone de los siguientes métodos, entre otros:

```
void dibujarRectangulo(double x, double y, double ancho, double alto, double angulo, Color color)
    ⇨ Dibuja un rectángulo centrado en el punto (x,y) de la pantalla, rotado en el ángulo dado.

void dibujarTriangulo(double x, double y, int altura, int base, double angulo, Color color)
    ⇨ Dibuja un triángulo centrado en el punto (x,y) de la pantalla, rotado en el ángulo dado.

void dibujarCirculo(double x, double y, double diametro, Color color)
    ⇨ Dibuja un círculo centrado en el punto (x,y) de la pantalla, del tamaño dado.

void dibujarImagen(Image imagen, double x, double y, double ang)
    ⇨ Dibuja la imagen centrada en el punto (x,y) de la pantalla rotada en el ángulo dado.

boolean estaPresionada(char t)
    ⇨ Indica si la tecla t está presionada por el usuario en ese momento.

boolean sePresiono(char t)
    ⇨ Indica si la tecla t fue presionada en este instante de tiempo (es decir, no estaba presionada en la última llamada a tick(), pero sí en ésta). Este método puede ser útil para identificar eventos particulares en un único momento, omitiendo tick() futuros en los cuales el usuario mantenga presionada la tecla en cuestión.

void escribirTexto(String texto, double x, double y)
    ⇨ Escribe el texto en las coordenadas x e y de la pantalla.

void cambiarFont(String font, int tamano, Color color)
    ⇨ Cambia la fuente para las próximas escrituras de texto según los parámetros recibidos.
```

Notar que los métodos `estaPresionada(char t)` y `sePresiono(char t)` reciben como parámetro un **char** que representa “la tecla” por la cual se quiere consultar, e.g., `sePresiono('A')` o `estaPresionada('+')`. Algunas teclas no pueden escribirse directamente como un **char** como por ejemplo las flechas de dirección del teclado. Para ellas, dentro de la clase **entorno** se encuentran las siguientes definiciones:

Valor	Tecla Representada
TECLA_ARRIBA	Flecha hacia arriba
TECLA_ABAJO	Flecha hacia abajo
TECLA_DERECHA	Flecha hacia la derecha
TECLA_IZQUIERDA	Flecha hacia la izquierda
TECLA_ENTER	Tecla “enter”
TECLA_ESPACIO	Barra espaciadora
TECLA_CTRL	Tecla “control”
TECLA_ALT	Tecla “alt”
TECLA_SHIFT	Tecla “shift”
TECLA_INSERT	Tecla “ins”
TECLA_DELETE	Tecla “del” (o “supr”)
TECLA_INICIO	Tecla “start” (o “inicio”)
TECLA_FIN	Tecla “end” (o “fin”)

De esta manera, para ver, por ejemplo, si el usuario está presionando la flecha hacia arriba se puede consultar, por ejemplo, si `estaPresionada(entorno.TECLA_ARRIBA)`.

Para más especificaciones del proyecto base ver el documento Especificaciones del Entorno en el moodle.